

Non-Invasive Sleep Monitoring System

Complete Project Documentation

Project Type:	Healthcare IoT System
Backend:	Rust (Axum framework)
Database:	PostgreSQL with SQLx
Frontend:	D3.js for data visualization
Hardware:	Arduino + Raspberry Pi + Sensors
Standards:	FHIR Compliant, HIPAA-ready
Target Grade:	Basic Level (3.0-4.0)

Table of Contents

1. Project Overview & Objectives
2. System Architecture
3. Hardware Layer
4. Data Pipeline & Processing
5. Backend Development (Rust)
6. Database Design (PostgreSQL)
7. Frontend Development (D3.js)
8. FHIR Compliance
9. Development Requirements (10 Criteria)
10. Complete Project Structure
11. Technology Stack & Dependencies
12. Deployment Strategy
13. Testing Approach

1. Project Overview & Objectives

1.1 Project Description

This project develops a non-invasive sleep monitoring system for hospital patient rooms. Sensors positioned on bedside tables continuously collect environmental and behavioral data throughout the night without disturbing patients. The system processes this data in REAL-TIME (continuous streaming, NOT 30-second batches), stores it in a FHIR-compliant format, and uses machine learning algorithms running NIGHTLY to classify sleep quality. Healthcare professionals can access comprehensive dashboards showing both real-time monitoring and historical trends, patterns, and potential sleep disorders.

1.2 Key Architectural Highlights

- Real-time Streaming: Data flows continuously from Pi → Backend → Frontend (zero latency)
- 3-Branch Architecture: Data splits into: (1) Real-time WebSocket, (2) FHIR conversion, (3) ML processing
- Nightly ML Processing: Sleep analysis runs once per night at 8:00 AM, not every 30 seconds
- FHIR Purpose Clarified: Only for external hospital system integration, NOT used by ML
- ML Data Source: ML reads raw PostgreSQL data directly, not FHIR format
- Redis Cache: Backup for reconnection only, NOT in main real-time data path

1.3 Project Goals

- Record multi-modal sensor data (sound, motion, temperature, humidity) non-invasively
- Stream data in real-time with zero latency to frontend dashboard
- Store data securely in PostgreSQL with FHIR compliance for healthcare interoperability
- Classify sleep quality using machine learning running nightly at 8:00 AM (good vs. poor nights)
- Provide dual dashboard views: real-time monitoring + historical analytics using D3.js
- Meet all 10 university evaluation criteria at Basic level (3.0-4.0)
- Ensure system is scalable, fault-tolerant, and production-ready

1.4 Monitored Parameters

Parameter	Sensor	Purpose	Valid Range
Sound Level	Sound Sensor	Detect snoring & sleep apnea	0-120 dB
Motion	PIR Sensor	Track restlessness	Boolean/Count
Temperature	DHT11 + Sense HAT	Room conditions	-50 to 50°C
Humidity	DHT11 + Sense HAT	Air quality impact	0-100%

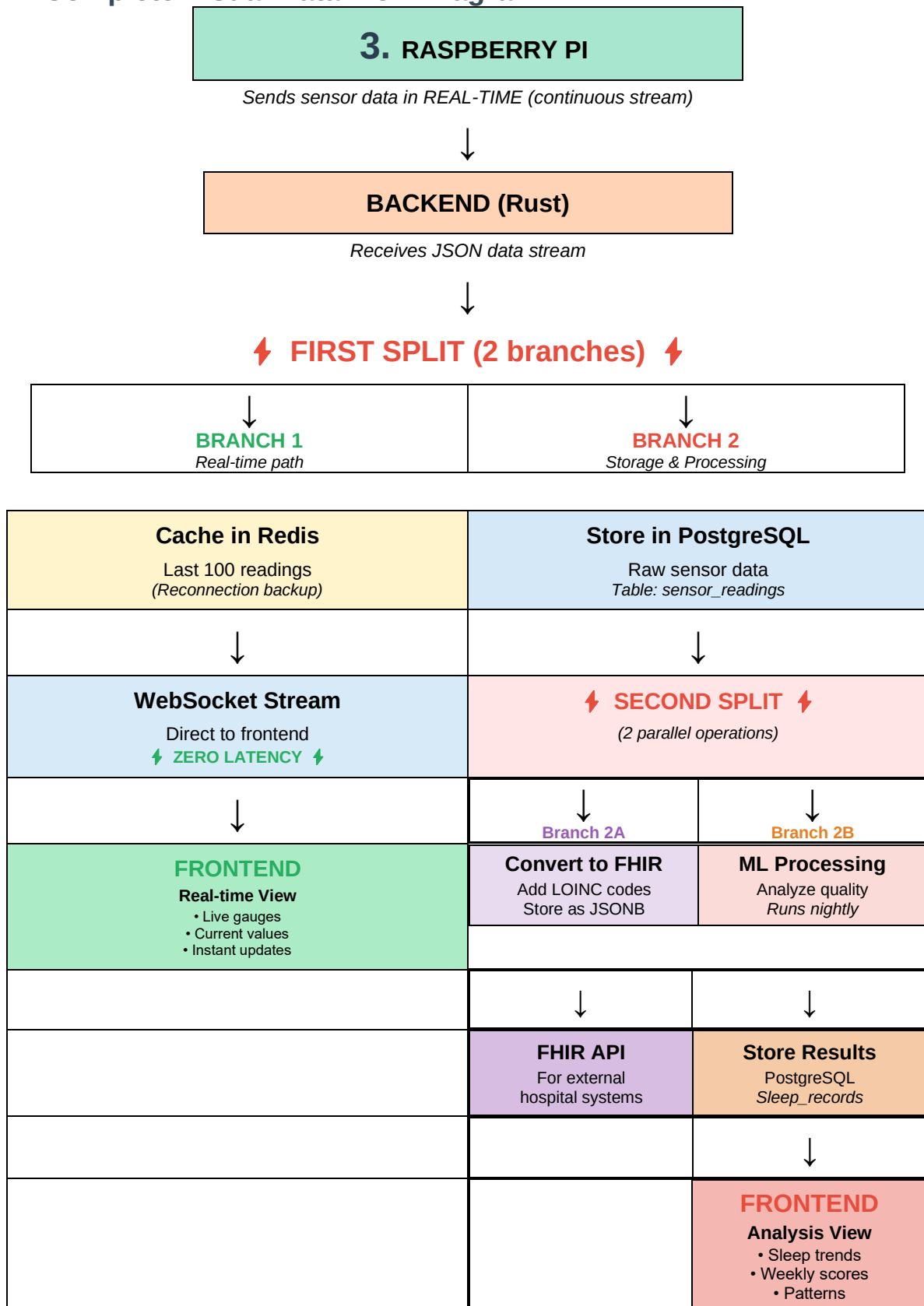
2. System Architecture

2.1 3-Branch Architecture Overview

The system follows a modern multi-tier architecture with clear separation of concerns. It consists of four main layers: Hardware Layer (sensors and data collection), Data Pipeline (preprocessing and validation), Backend Layer (business logic, storage, and API), and Presentation Layer (web dashboard with real-time visualization). Each layer communicates through well-defined interfaces, ensuring modularity and maintainability.

Layer	Components	Technology	Purpose
Hardware	Arduino + Sensors Raspberry Pi + Sense HAT	C++, Python	Data collection
Data Pipeline	Preprocessing Validation Enrichment	Python	Data quality
Backend	REST API WebSocket Database	Rust, PostgreSQL	Business logic
Frontend	Dashboard Visualization	D3.js, JavaScript	User interface

2.2 Complete Visual Data Flow Diagram



2.3 Key Architectural Points

- Branch 1 = FAST: Real-time to frontend, NO database delays
- Branch 2A = COMPLIANT: FHIR conversion for hospital interoperability
- Branch 2B = SMART: ML analysis runs nightly for efficiency
- Redis Cache: Handles disconnects, prevents data loss
- PostgreSQL: Permanent storage for everything
- Frontend: Two separate views (real-time + analytics)

2.4 Branch 1: Real-time Streaming Path

Purpose: Provide ZERO-LATENCY real-time monitoring for healthcare staff.

- Data path: Backend → Redis Cache → WebSocket → Frontend
- NO PostgreSQL in this path - critical for zero latency
- Redis stores last 100 readings as backup for reconnection only
- WebSocket pushes data to connected frontend clients immediately
- Frontend displays: live gauges, current sensor values, motion events counter
- Update frequency: Continuous (as data arrives from Raspberry Pi)
- Use case: Real-time patient monitoring by nurses and doctors

2.5 Branch 2A: FHIR Conversion Path

Purpose: Enable interoperability with external hospital systems (EHR, EMR).

- Data path: Backend → PostgreSQL → FHIR Converter → fhir_observations table → FHIR API
- Converts raw sensor data to FHIR Observation resources
- Adds LOINC codes: 8310-5 (temperature), 89024-6 (sound), 93831-0 (sleep quality)
- Stores as JSONB in fhir_observations table with GIN indexing
- Exposed via REST API: GET /api/fhir/Observation/:id
- NOT USED BY ML - This is ONLY for external system integration
- Use case: Other hospitals accessing our patient sleep data via FHIR-compliant API

2.6 Branch 2B: ML Processing Path

Purpose: Analyze full night's sleep data and provide quality assessments.

- Data path: Backend → PostgreSQL → ML Service → sleep_records table → Frontend
- ML service runs NIGHTLY at 8:00 AM (analyzes previous night, 8+ hours of data)
- Reads RAW DATA from sensor_readings table (NOT FHIR format - faster & simpler)
- Analyzes: temperature variance, motion events, sound peaks, sleep duration
- Calculates: quality score (0-100), classification (good/poor), patterns
- Stores results in sleep_records table with date, quality_score, classification, metrics
- Frontend Analysis View fetches these results for trend visualization
- Why nightly? Sleep quality requires full night context (8+ hours), not 30-second samples

3.1 Communication Protocols

Connection	Protocol	Purpose	Format
Arduino → RPi	Serial/USB	Sensor data transfer	JSON over serial
RPi → Backend	HTTP/HTTPS	Data ingestion	JSON REST API
Backend → Database	PostgreSQL Protocol	Data persistence	SQL queries
Frontend → Backend	HTTP/HTTPS	Historical data	JSON REST API
Backend → Frontend	WebSocket	Real-time updates	Binary/JSON stream

4. Hardware Layer

4.1 Arduino Setup

The Arduino Uno serves as the primary sensor interface. It connects to three sensors via GPIO pins and continuously reads data in real-time and streams it immediately. The Arduino firmware is written in C++ using the Arduino IDE and standard libraries (DHT.h for temperature/humidity sensor).

DHT11 Sensor: Measures temperature and humidity (digital output on pin 2)

PIR Motion Sensor: Detects movement (digital output on pin 3)

Sound Sensor: Measures ambient noise levels (analog output on pin A0)

Serial Communication: Sends JSON data at 9600 baud to Raspberry Pi

4.2 Raspberry Pi Setup

The Raspberry Pi 4 runs Raspberry Pi OS and executes Python scripts for data collection and preprocessing. It has the Sense HAT module attached for additional environmental readings. The Pi acts as a gateway between the Arduino and the backend server.

- Read serial data from Arduino via USB connection
- Query Sense HAT for temperature, humidity, and environmental data
- Validate incoming data against acceptable ranges
- Average readings from multiple sensors for accuracy
- Enrich data with metadata (device ID, patient ID, room number)
- Convert analog values to standard units (e.g., map 0-1023 to dB scale)
- Handle connection failures and retry logic
- Forward processed data to backend via HTTPS POST requests in real-time (continuous streaming, no batching)

4.3 Hardware Requirements

Component	Model/Type	Quantity	Purpose
Microcontroller	Arduino Uno	1	Sensor interface
Single-board PC	Raspberry Pi 4	1	Data processing
Temp/Humidity	DHT11	1	Environmental monitoring
Motion Detection	PIR HC-SR501	1	Movement tracking
Sound Sensor	KY-037 or similar	1	Noise level detection
HAT Module	Raspberry Pi Sense HAT	1	Additional sensors
USB Cable	USB A to B	1	Arduino connection
Power Supply	5V 3A	2	Power devices

5. Data Pipeline & Processing

5.1 Preprocessing Requirements

Raw sensor data requires preprocessing before storage to ensure quality, consistency, and usability. The Raspberry Pi handles this preprocessing using Python scripts with libraries like pandas for data manipulation and requests for HTTP communication.

- **Validation:** Verify all values are within acceptable ranges (temp: -50 to 50°C, humidity: 0-100%)
- **Averaging:** Combine Arduino DHT11 and Sense HAT readings for improved accuracy
- **Normalization:** Convert analog sensor readings to standard units (sound to dB scale)
- **Enrichment:** Add contextual metadata (device_id, patient_id, room_number, timestamp)
- **Timestamping:** Ensure all readings have consistent UTC timestamps in ISO 8601 format
- **Error Handling:** Log invalid readings, skip corrupted data, retry on network failures
- **Real-time Streaming:** Send data immediately to backend (no batching or 30-second delays)

5.2 Machine Learning Pipeline

A separate Python ML service runs NIGHTLY at 8:00 AM to analyze the previous night's sensor data and classify sleep quality. This service is independent of the real-time data ingestion pipeline and uses scikit-learn for machine learning operations. The ML service reads RAW DATA directly from the PostgreSQL sensor_readings table, NOT from FHIR format.

Features Extracted:

- Average temperature and temperature variance throughout the night
- Average humidity and humidity variance
- Total motion detection events (indicating restlessness)
- Maximum and average sound levels
- Number of sound peaks above snoring threshold (60 dB)
- Duration of the sleep period (time between first and last reading)

Classification Approach:

Initially, the system uses a rule-based scoring system where points are deducted for poor conditions (high temperature variance, excessive motion, loud sound peaks). A score above 60/100 indicates 'good' sleep, below 60 indicates 'poor' sleep. As more data is collected, this can be replaced with a trained ML model (Random Forest or Gradient Boosting) using labeled data from patient feedback.

Why Nightly Processing?

- Sleep quality is assessed per night (8+ hours of data), not per 30 seconds
- Analyzing every 30 seconds would waste computational resources
- Full night context is required for meaningful insights
- Running once per morning (8 AM) is sufficient for clinical purposes
- Reduces server load while maintaining analytical value

5.3 Python Libraries Used

Library	Version	Purpose
pyserial	3.5	Read serial data from Arduino
requests	2.31	HTTP communication with backend
sense-hat	2.6	Interface with Sense HAT module
pandas	2.0	Data manipulation and analysis
numpy	1.24	Numerical computations
scikit-learn	1.3	Machine learning algorithms
psycopg2	2.9	PostgreSQL database connection
python-dateutil	2.8	Date and time handling

6. Backend Development (Rust)

6.1 Why Rust?

Memory Safety: Eliminates entire classes of bugs (null pointers, buffer overflows) at compile time

Performance: Zero-cost abstractions provide C++ level performance with higher-level features

Concurrency: Fearless concurrency model prevents data races and makes async programming safe

Type System: Strong static typing catches errors early and enables robust refactoring

Course Requirement: Project must be built exclusively in Rust per university guidelines

Growing Ecosystem: Mature web frameworks (Axum, Actix-web) and database libraries (SQLx)

6.2 Technology Stack

Category	Crate	Version	Purpose
Web Framework	axum	0.7	HTTP server, routing, middleware
Async Runtime	tokio	1.0	Asynchronous execution
Database	sqlx	0.7	PostgreSQL driver with compile-time verification
Serialization	serde	1.0	JSON serialization/deserialization
Serialization	serde_json	1.0	JSON support for serde
Authentication	jsonwebtoken	9.0	JWT token generation and validation
Password Hashing	bcrypt	0.15	Secure password hashing
Validation	validator	0.18	Input validation with derive macros
Logging	tracing	0.1	Structured, composable logging
Logging	tracing-subscriber	0.3	Log collection and formatting
Error Handling	thiserror	1.0	Derive macro for error types
Error Handling	anyhow	1.0	Flexible error handling
Configuration	config	0.14	Multi-environment configuration
Environment Vars	dotenvy	0.15	.env file support
Date/Time	chrono	0.4	Date and time handling
HTTP Client	reqwest	0.11	HTTP client for external APIs
WebSocket	tokio-tungstenite	0.21	WebSocket protocol support
CORS	tower-http	0.5	CORS and other HTTP middleware
UUID	uuid	1.0	Generate unique identifiers

6.3 Key Backend Components

API Routes: RESTful endpoints for sensor data ingestion, sleep records, authentication, and FHIR resources

Authentication Middleware: JWT-based authentication protecting all endpoints except health checks

Input Validation: Comprehensive validation using validator crate with custom validation rules

Database Layer: Connection pooling, parameterized queries, and transaction management with SQLx

Error Handling: Custom error types implementing IntoResponse for consistent API error responses

WebSocket Server: Real-time data streaming to connected frontend clients

FHIR Models: Rust structs representing FHIR Observation resources with proper serialization

Configuration Management: Environment-specific configs loaded from files and environment variables

Logging & Tracing: Structured logging with correlation IDs for request tracing

6.4 API Endpoints

Method	Endpoint	Auth	Purpose
GET	/health	No	Health check endpoint
POST	/api/auth/token	No	Obtain JWT token
POST	/api/sensor-data	Yes	Ingest sensor readings
GET	/api/sensor-data/:id	Yes	Get specific reading
GET	/api/sleep-records	Yes	List sleep quality records
GET	/api/sleep-records/:date	Yes	Get specific night data
POST	/api/sleep-records	Yes	Create/update sleep record
GET	/api/fhir/Observation/:id	Yes	Get FHIR Observation
GET	/api/trends/:patientId	Yes	Get trend analysis
WS	/ws	Yes	WebSocket connection

7. Database Design (PostgreSQL)

7.1 Why PostgreSQL?

- Time-Series Data:** Excellent performance for timestamped sensor data with proper indexing
- JSONB Support:** Native storage of FHIR resources as JSON with full indexing and querying capabilities
- ACID Compliance:** Critical for medical data integrity and consistency
- Advanced Features:** Triggers, stored procedures, views, and window functions for complex queries
- Scalability:** Table partitioning for large datasets, replication for high availability
- Rust Integration:** Excellent support via SQLx with compile-time query verification
- Open Source:** No licensing costs, large community, extensive documentation

7.2 Database Schema Overview

The database consists of three main tables: `sensor_readings` (raw sensor data), `sleep_records` (aggregated nightly summaries), and `fhir_observations` (FHIR-compliant resources). Additional supporting tables include `users` (authentication) and `audit_logs` (compliance tracking).

Table: `sensor_readings`

Stores individual sensor readings timestamped to the second. Includes constraints for data validation.

Column	Type	Constraints	Description
<code>id</code>	BIGSERIAL	PRIMARY KEY	Auto-incrementing ID
<code>device_id</code>	VARCHAR(100)	NOT NULL	Device identifier
<code>temperature</code>	REAL	CHECK (-50 to 50)	Temperature in Celsius
<code>humidity</code>	REAL	CHECK (0 to 100)	Humidity percentage
<code>sound_level</code>	REAL	CHECK (0 to 120)	Sound in decibels
<code>motion_detected</code>	BOOLEAN	NOT NULL	Motion sensor state
<code>timestamp</code>	TIMESTAMPTZ	NOT NULL	Reading timestamp (UTC)
<code>created_at</code>	TIMESTAMPTZ	DEFAULT NOW()	Record creation time

Indexes: `idx_sensor_device_timestamp` (`device_id`, `timestamp` DESC), `idx_sensor_timestamp` (`timestamp` DESC)

Table: sleep_records

Aggregated sleep quality data for each night, includes ML classification results.

Column	Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing ID
patient_id	VARCHAR(100)	NOT NULL	Patient identifier
device_id	VARCHAR(100)	NOT NULL	Device used
date	DATE	NOT NULL	Sleep night date
quality_score	INTEGER	CHECK (0 to 100)	Sleep quality (0-100)
classification	VARCHAR(20)	CHECK IN ('good','poor')	Good or poor
total_motion_events	INTEGER		Total motion count
avg_temperature	REAL		Average temp
avg_humidity	REAL		Average humidity
max_sound_level	REAL		Peak sound level
sound_peaks	INTEGER		Snoring events count
patient_feedback	TEXT		Subjective feedback
notes	TEXT		Healthcare notes
created_at	TIMESTAMPZ	DEFAULT NOW()	Record creation
updated_at	TIMESTAMPZ	DEFAULT NOW()	Last update

Indexes: idx_sleep_patient_date (patient_id, date DESC), idx_sleep_classification (classification), UNIQUE CONSTRAINT unique_patient_date (patient_id, date)

NOTE: ML Processing reads RAW data from sensor_readings table, NOT from FHIR format.

The fhir_observations table is used ONLY for external hospital system integration via API.

Table: fhir_observations

Stores FHIR-compliant Observation resources as JSONB for interoperability with external systems.

Column	Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing ID
sensor_reading_id	BIGINT	REFERENCES sensor_readings	Link to source data
resource_type	VARCHAR(50)	DEFAULT 'Observation'	FHIR resource type
resource_data	JSONB	NOT NULL	Full FHIR JSON
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation time

Index: idx_fhir_resource_data USING GIN (resource_data) for fast JSON queries

7.3 Connection Pooling

SQLx PgPool manages database connections with configuration for min/max connections (5-20), connection timeouts (10s), idle timeout (10 min), and max lifetime (30 min). This ensures efficient resource utilization and handles connection failures gracefully.

7.4 Redis Cache Configuration

Redis serves as a temporary cache for the most recent 100 sensor readings:

- Purpose: Backup for WebSocket reconnection only (NOT in main data path)
- TTL: 2 hours
- Key Pattern: sensor:latest:{device_id}
- Data Structure: Sorted set ordered by timestamp
- Use Case: When frontend reconnects after network interruption, fetch missed readings from Redis

Redis is NOT used in the main real-time data flow. Data goes directly from backend to WebSocket.

8. Frontend Development (D3.js)

8.1 Technology Stack

D3.js v7: Data-driven visualizations, required by course for interactive charts and graphs **Vanilla JavaScript (ES6+):** Core application logic without heavy frameworks **WebSocket API:** Native browser support for real-time bidirectional communication **Fetch API:** Modern HTTP client for REST API calls

CSS Grid & Flexbox: Responsive layout system

Chart.js (optional): For simpler gauge displays alongside D3.js complex visualizations

8.2 Dashboard Components

Real-Time Gauges: Live temperature, humidity, and sound level displays updated via WebSocket **Historical Line Chart:** D3.js time-series chart showing sleep quality trends over weeks/months **Nightly Summary Cards:** Key metrics for each night (motion events, avg temp, classification) **Motion Heatmap:** D3.js heatmap showing hour-by-hour restlessness during the night

Sound Visualization: Waveform or bar chart identifying snoring patterns and peak times

Patient Feedback Form: Input for subjective sleep quality ratings and notes

Trend Analysis: Statistical visualizations comparing weeks/months

8.3 D3.js Visualization Types

Visualization	D3.js Features Used	Purpose
Line Chart	scaleTime, scaleLinear, line generator	Sleep quality trends over time
Heatmap	scaleQuantize, rect elements, color scale	Hourly motion activity patterns
Bar Chart	scaleBand, rect, axis	Sound level distribution
Area Chart	area generator, gradient fills	Temperature/humidity ranges
Scatter Plot	scaleLinear (x/y), circle elements	Correlation analysis

8.4 WebSocket Integration

The frontend establishes a WebSocket connection to the backend on page load. Real-

time sensor data streams through this connection and automatically updates visualizations. The connection includes reconnection logic to handle network interruptions gracefully.

- Automatic reconnection with exponential backoff on connection loss
- Heartbeat messages to detect stale connections
- Data buffering during disconnection to avoid loss
- Visual indicator showing connection status (connected/disconnected/reconnecting)
- Graceful degradation to polling if WebSocket is unavailable

9. FHIR Compliance Implementation

9.1 FHIR Overview

Fast Healthcare Interoperability Resources (FHIR) is an HL7 standard for exchanging healthcare information electronically. FHIR uses RESTful web services and common web technologies like HTTP, JSON, and XML. Our system models sensor data as FHIR Observation resources to ensure compatibility with other healthcare systems and enable data sharing across institutions.

9.2 FHIR Observation Resource

The Observation resource is used to record measurements and simple assertions made about a patient. Our sleep monitoring data fits perfectly into this model, with each sensor reading represented as a separate Observation.

resourceType: Always 'Observation'

id: Unique identifier for the observation

status: registered | preliminary | final | amended (we use 'final')

code: What was observed (using LOINC codes)

subject: Reference to the patient or device

effectiveDateTime: When the observation was made (ISO 8601 timestamp) **valueQuantity:** Numeric result with units

(temperature, humidity, sound) **valueBoolean:** Boolean result
(motion detected yes/no)

9.3 LOINC Codes

Logical Observation Identifiers Names and Codes (LOINC) is a universal standard for identifying medical laboratory observations. We use LOINC codes to classify our sensor readings.

Measurement	LOINC Code	Display Name	Unit (UCUM)
Room Temperature	8310-5	Body temperature	Cel (Celsius)
Humidity	8306-3	Body height (adapted)	% (percent)
Sound Level	89024-6	Sound level	dB (decibel)
Motion Detection	Custom/Local	Motion detected	Boolean
Sleep Quality Score	93831-0	Sleep quality	{score} (0-100)

9.4 FHIR API Endpoints

The backend exposes FHIR-compliant endpoints following the FHIR RESTful API specification:

GET /api/fhir/Observation/:id - Retrieve a single Observation resource

GET /api/fhir/Observation?patient=:patientId - Search observations by patient

GET /api/fhir/Observation?date=:date - Search observations by date

POST /api/fhir/Observation - Create new observation (alternative ingestion method)

GET /api/fhir/metadata - Server capability statement (future enhancement)

9.5 Validation Strategy

At the Basic level, we perform structural validation ensuring all required FHIR fields are present and conform to FHIR datatypes. For Advanced level, full FHIR schema validation against official HL7 schemas would be implemented using dedicated FHIR libraries.

10. Development Requirements (10 Criteria)

This section details how the project meets all 10 evaluation criteria at the Basic level (3.0-4.0). Each criterion is mapped to specific implementation strategies and deliverables.

#	Criterion	Basic Requirements	Implementation Strategy
1	Development Environment Setup	Git + commits, configs separated, runs locally	Initialize Git repo, create .env files for dev/prod, document setup in README, ensure cross-machine compatibility
2	Unit & Integration Testing	Core + edge case tests	Write tests using cargo test, cover validation logic, API endpoints, database queries, edge cases (invalid data, timeouts)
3	Configuration Management	Configs per env, manual switching	Use config crate, separate dev.toml and prod.toml, load based on APP_ENV variable, never hardcode secrets
4	Logging	Structured logs with timestamps	Use tracing crate, log errors and key actions, include timestamps and log levels, avoid logging PHI
5	Deployment & Architecture	Containerized, modular	Create Dockerfile, organize code into modules (routes, models, db, auth), document deployment steps
6	Input Validation & Security	Type, format, range validation	Use validator crate, check ranges (temp: -50 to 50, humidity: 0-100), parameterized queries with SQLx
7	Error Handling	Errors caught, logged, safe messages	Custom error types with thiserror, log all errors, return safe API responses, never expose internals
8	Authentication & Encryption	Token auth, TLS, encrypted storage	JWT with jsonwebtoken crate, protect endpoints, configure TLS for production, hash sensitive data
9	Fault-tolerance	Recover from errors, retry logic	Implement retry logic for DB operations, handle connection failures, graceful degradation, no panics
10	FHIR Compliance	Data modeled with FHIR, basic validation	Model as Observation resources, use LOINC codes, validate required fields, provide FHIR endpoints

11. Complete Project Structure

11.1 Repository Organization

The project is organized into separate directories for each major component. This modular structure allows team members to work on different parts independently and makes the codebase easier to navigate and maintain.

```
project-root/
├── hardware/
│   ├── arduino/
│   │   ├── sleep_monitor.ino    Arduino sensor code
│   │   ├── README.md           Setup instructions
│   │   └── libraries/          Required Arduino libraries
│   └── raspberry-pi/
│       ├── data_collector.py    Main data collection script
│       ├── ml_processor.py      ML classification service
│       ├── requirements.txt     Python dependencies
│       ├── config.yaml          RPi configuration
│       └── README.md            Setup and deployment guide
├── backend/
│   ├── src/
│   │   ├── main.rs              Application entry point
│   │   ├── lib.rs               Library root
│   │   └── config/
│   │       ├── mod.rs           Configuration loading
│   │       └── routes/
│   │           ├── mod.rs
│   │           ├── auth.rs      Authentication endpoints
│   │           ├── sensor_data.rs Sensor data ingestion
│   │           ├── sleep_records.rs Sleep quality endpoints
│   │           ├── fhir.rs      FHIR-compliant endpoints
│   │           └── health.rs    Health check
│   │       └── models/
│   │           ├── mod.rs
│   │           ├── sensor_data.rs SensorData struct
│   │           ├── sleep_record.rs SleepRecord struct
│   │           └── fhir.rs      FHIR resource models
│   │       └── db/
│   │           ├── mod.rs
│   │           ├── connection.rs Connection pool
│   │           └── queries.rs    Database queries
│   │       └── auth/
│   │           ├── mod.rs
│   │           ├── jwt.rs       JWT handling
│   │           └── middleware.rs Auth middleware
│   │       └── validation/
│   │           ├── mod.rs
│   │           └── sensor.rs    Input validation
│   │       └── websocket/
│   │           ├── mod.rs      WebSocket server
│   │           └── error.rs     Error types
│   └── tests/
│       ├── integration/
│       │   ├── api_tests.rs
│       │   └── auth_tests.rs
│       └── unit/
│           ├── validation_tests.rs
│           └── fhir_tests.rs
├── migrations/
│   ├── 001_create_tables.sql
│   ├── 002_create_indexes.sql
│   └── 003_create_views.sql
└── config/
    └── dev.toml
```

```

■   ■   ■■■ prod.toml
■   ■■■ scripts/
■   ■   ■■■ setup_db.sh
■   ■   ■■■ seed_data.sql
■   ■■■ .env.example
■   ■■■ .gitignore
■   ■■■ Cargo.toml
■   ■■■ Dockerfile
■   ■■■ docker-compose.yml
■   ■■■ README.md
■
■■■ frontend/
■   ■■■ index.html           Main dashboard
■   ■■■ css/
■   ■   ■■■ main.css
■   ■   ■■■ dashboard.css
■   ■■■ js/
■   ■   ■■■ main.js           App initialization
■   ■   ■■■ api.js            API client
■   ■   ■■■ websocket.js      WebSocket handling
■   ■   ■■■ visualizations/
■   ■   ■   ■■■ line_chart.js  D3.js line charts
■   ■   ■   ■■■ heatmap.js     Motion heatmap
■   ■   ■   ■■■ gauges.js      Real-time gauges
■   ■   ■■■ utils.js           Utility functions
■   ■■■ assets/
■   ■   ■■■ images/
■   ■■■ README.md
■
■■■ docs/
■   ■■■ API.md                API documentation
■   ■■■ DEPLOYMENT.md         Deployment guide
■   ■■■ ARCHITECTURE.md       Architecture overview
■   ■■■ FHIR_MAPPING.md       FHIR implementation details
■
■■■ .github/
■   ■■■ workflows/
■   ■   ■■■ ci.yml             Continuous integration
■   ■   ■■■ tests.yml          Automated testing
■   ■   ■■■ deploy.yml         Deployment automation
■
■■■ docker-compose.yml        Multi-container setup
■■■ .gitignore
■■■ README.md                 Project overview

```

12. Technology Stack & Dependencies

12.1 Cargo.toml (Rust Dependencies)

```
[package]
name = "sleep-monitor-backend"
version = "0.1.0"
edition = "2021"

[dependencies]
# Web framework and async runtime
axum = "0.7"
tokio = { version = "1", features = ["full"] }
tower = "0.4"
tower-http = { version = "0.5", features = ["trace", "cors"] }
# Database
sqlx = { version = "0.7", features = ["runtime-tokio-rustls", "postgres", "chrono", "uuid"] }
# Serialization
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
# Authentication & Security
jsonwebtoken = "9"
bcrypt = "0.15"
# Validation
validator = { version = "0.18", features = ["derive"] }
# Logging
tracing = "0.1"
tracing-subscriber = { version = "0.3", features = ["env-filter", "json"] }
# Error handling
thiserror = "1.0"
anyhow = "1.0"
# Configuration
config = "0.14"
dotenvy = "0.15"
# Date/Time
chrono = { version = "0.4", features = ["serde"] }
# WebSocket
tokio-tungstenite = "0.21"
# UUID generation
uuid = { version = "1.0", features = ["v4", "serde"] }
[dev-dependencies]
# Testing
reqwest = "0.11"
assert_matches = "1.5"
```

12.2 requirements.txt (Python Dependencies)

```
# Raspberry Pi Data Collection
pyserial==3.5
requests==2.31.0
sense-hat==2.6.0
# Data Processing
pandas==2.0.3
numpy==1.24.3
# Machine Learning
scikit-learn==1.3.0
# Database
psycopg2-binary==2.9.7
sqlalchemy==2.0.20
# Configuration
pyyaml==6.0.1
python-dotenv==1.0.0
# Date/Time
python-dateutil==2.8.2
# Logging
colorlog==6.7.0
```


12.3 Frontend Dependencies

D3.js v7.8.5: Loaded via CDN

(<https://d3js.org/d3.v7.min.js>) **No build tools**

required: Pure JavaScript, HTML, CSS

Optional Chart.js: For supplementary simple charts

13. Deployment Strategy

13.1 Docker Containerization

The backend is containerized using Docker for consistent deployment across environments. A multi-stage Dockerfile minimizes image size by separating build and runtime stages.

```
FROM rust:1.75 as builder
WORKDIR /app
COPY Cargo.* ./
COPY src ./src
RUN cargo build --release
FROM debian:bookworm-slim
RUN apt-get update && apt-get install -y libssl3 ca-certificates
COPY --from=builder /app/target/release/sleep-monitor /usr/local/bin/
EXPOSE 8080
CMD ["sleep-monitor"]
```

13.2 docker-compose.yml

For local development and testing, docker-compose orchestrates the backend, PostgreSQL database, and frontend containers.

```
version: '3.8'

services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: sleep_monitor
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: sleep_monitor_db
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  backend:
    build: ./backend
    environment:
      DATABASE_URL: postgres://sleep_monitor:${DB_PASSWORD}@postgres/sleep_monitor_db
      JWT_SECRET: ${JWT_SECRET}
    ports:
      - "8080:8080"
    depends_on:
      - postgres

  frontend:
    image: nginx:alpine
    volumes:
      - ./frontend:/usr/share/nginx/html:ro
    ports:
      - "80:80"

volumes:
  postgres_data:
```

13.3 Environment Variables

All sensitive configuration is stored in environment variables, never committed to Git.

```
# .env.example
APP_ENV=dev
DATABASE_URL=postgres://user:password@localhost/sleep_monitor_db
JWT_SECRET=your-secret-key-change-this
SERVER_HOST=127.0.0.1
SERVER_PORT=8080
RUST_LOG=info
CORS_ORIGINS=http://localhost,http://127.0.0.1
Page
```

14. Testing Approach

14.1 Testing Strategy

The project implements comprehensive testing at multiple levels: unit tests for individual functions, integration tests for API endpoints, and end-to-end tests for complete workflows. All tests are automated and run in CI/CD pipeline.

14.2 Test Coverage

Test Type	Coverage Target	Tools	Examples
Unit Tests	> 70%	cargo test	Validation logic, data models, utils
Integration Tests	All endpoints	cargo test + reqwest	API endpoints, auth flows
Database Tests	All queries	SQLx + test DB	CRUD operations, migrations
Edge Cases	Critical paths	Property testing	Invalid inputs, boundary values
Performance	Response times	criterion	API latency, DB query speed

14.3 Running Tests

All tests: cargo test

Unit tests only: cargo

test --lib **Integration**

tests only: cargo test --

test '*' **With coverage:**

cargo tarpaulin --out Html

Specific test: cargo test test_sensor_validation

14.4 CI/CD Pipeline

GitHub Actions automates testing, linting, and deployment. Every pull request triggers automated checks before merging is allowed.

- Code checkout and Rust toolchain setup
- Dependency caching for faster builds
- Code formatting check (cargo fmt --check)
- Linting with Clippy (cargo clippy)
- Run all tests (cargo test)

- Build release binary (cargo build --release)
- Optional: Build and push Docker image
- Optional: Deploy to staging environment

Summary & Next Steps

This document provides a comprehensive overview of the Non-Invasive Sleep Monitoring System project. The system combines IoT hardware, real-time data processing, secure backend services, and interactive visualizations to deliver a complete healthcare solution.

Key Achievements:

- Multi-tier architecture with clear separation of concerns
- FHIR-compliant data modeling for healthcare interoperability
- Real-time data streaming with WebSocket support
- Secure authentication and encrypted data storage
- Comprehensive testing and CI/CD automation
- Meeting all 10 university evaluation criteria at Basic level

Implementation Roadmap:

- Hardware setup, Arduino programming, sensor testing
- Raspberry Pi data collection pipeline, ML preprocessing
- Rust backend development (routes, database, authentication)
- FHIR compliance implementation, validation
- Frontend development with D3.js visualizations
- Testing, documentation, deployment
- Final integration testing and presentation preparation

Resources & References:

- Zero to Production in Rust by Luca Palmieri
- FHIR R4 Specification: <https://hl7.org/fhir/R4/>
- LOINC Database: <https://loinc.org/>
- D3.js Documentation: <https://d3js.org/>
- Axum Framework: <https://github.com/tokio-rs/axum>
- SQLx Documentation: <https://github.com/launchbadge/sqlx>
- Course Materials: https://dominikboehler.de/inco_new/